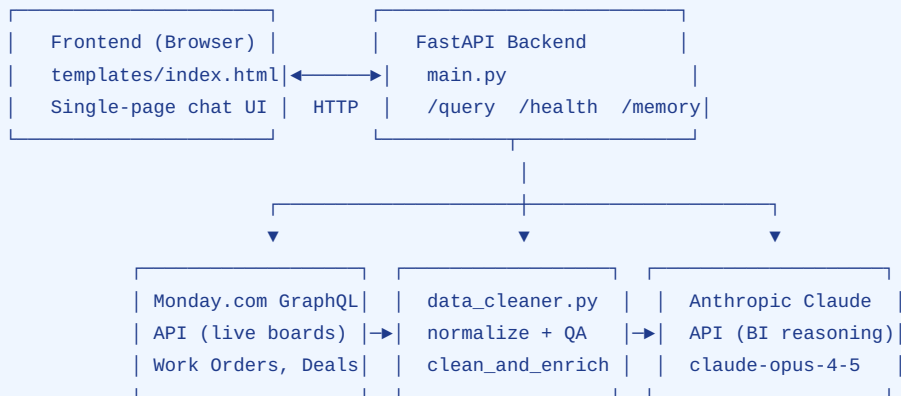


Monday.com BI Agent

Complete File Structure & System Architecture

Repository: github.com/Krish-Jain-IITJ/skylark_drones_ps_re · **Live:** skylark-drones-ps-re.vercel.app · **Stack:** Python / FastAPI / HTML-CSS-JS

1 High-Level Architecture



Request flow: a user question hits `POST /query` → the backend fetches fresh data from both Monday.com boards via GraphQL → `data_cleaner.py` normalizes and enriches the raw records (nulls, currency, dates) and produces a quality report → the cleaned dataset, quality report, and last 5 conversation turns are assembled into a system prompt → Claude analyzes it and returns a structured, analyst-style answer → the full API/tool trace is returned alongside the answer for transparency in the UI.

2 Complete File Structure

```
skylark_drones_ps_re/
|
├─ main.py                # FastAPI app – routes, Monday.com GraphQL calls, Claude orchestration
├─ data_cleaner.py        # Data cleaning / normalization / enrichment for WO + Deals records
├─ quality_report.json    # Pre-computed data quality report (null counts, fixes applied)
├─ requirements.txt       # Python dependencies
├─ vercel.json            # Vercel serverless routing config
├─ specmatic.yaml        # Root Specmatic contract-testing config
├─ monday_bi_agent.code-workspace # VS Code workspace file
|
├─ .env.example           # Template for required environment variables
├─ .gitignore
├─ .vercelignore
|
├─ README.md             # Main project documentation
├─ README_SPECMATIC.md   # Specmatic-specific readme
├─ PROJECT_DOCUMENTATION.md # Extended project documentation
├─ IMPLEMENTATION_SUMMARY.md # Summary of implementation decisions
├─ DELIVERY_MANIFEST.md  # Deliverables checklist
├─ FINAL_VERIFICATION.md  # Final QA / verification notes
├─ CODING_ROUND_SUBMISSION.md # Assignment submission notes
├─ QUICKSTART_SPECMATIC.md # Quickstart guide for contract tests
```

```

├── SPECMATIC_INTEGRATION.md      # How Specmatic is wired into the project
├── VISUAL_DIFFS.md              # Before/after visual diff notes
├──
├── api/
│   └── index.py                  # Vercel serverless entrypoint – imports FastAPI app from main.py
├── static/
│   └── logo.png                  # App logo asset
├── templates/
│   └── index.html                # Single-page chat frontend (HTML/CSS/JS, no build step)
├── specmatic/                    # API contract testing (Specmatic) suite
│   ├── specmatic.yaml           # Specmatic test runner config
│   ├── openapi.yaml             # OpenAPI contract for the FastAPI endpoints
│   ├── employees.yaml           # Sample contract spec
│   ├── employee_details.yaml    # Sample contract spec
│   ├── resiliency_tests.yaml    # Resiliency / fuzz test definitions
│   ├──
│   ├── employee_details_examples/ # Example request/response fixtures
│   │   ├── update_employee_missing_department_400.json
│   │   ├── update_employee_missing_name_400.json
│   │   ├── update_employee_success.json
│   │   ├── update_employee_with_code_success.json
│   │   └── update_employee_wrong_type_400.json
│   ├──
│   ├── openapi_examples/        # Contract examples per real API endpoint
│   │   ├── get_health_all_configured.json
│   │   ├── get_health_groq_not_configured.json
│   │   ├── get_memory_new_session_empty.json
│   │   ├── post_query_monday_timeout_502.json
│   │   ├── post_query_no_session_id.json
│   │   └── post_query_pipeline_value.json
│   ├──
│   └── stubs/                    # Mock server stubs for contract-driven testing
│       ├── delete_memory_session_001.json
│       ├── get_health_success.json
│       ├── get_memory_session_001.json
│       ├── get_quality_report_success.json
│       ├── post_query_config_error_503.json
│       ├── post_query_pipeline_success.json
│       ├── post_query_upstream_error_502.json
│       ├── post_query_workorders_success.json
│       ├──
│       └── resiliency/           # Edge-case stubs (fuzzing, malformed input, injection)
│           ├── memory_missing_session_id_defaults.json
│           ├── query_empty_string_422.json
│           ├── query_extra_fields_ignored.json
│           ├── query_injection_attempt.json
│           ├── query_missing_content_type.json
│           ├── query_missing_optional_session_id.json
│           ├── query_null_required_field_422.json
│           ├── query_very_long_string_boundary.json
│           └── query_wrong_type_integer_422.json

```

3 Core Application Modules

main.py **FastAPI app**

The entrypoint and orchestration layer. Defines the FastAPI app, mounts `/static`, configures CORS, and exposes all API routes. Contains the Monday.com GraphQL helper (`monday_gql`), board-fetching logic (`fetch_board` , `get_live_data`), per-session in-memory

conversation storage, the Claude prompt-assembly functions (`build_memory_ctx` , `format_quality_summary` , `ask_llm`), and the live API/tool trace that's returned to the frontend on every query.

`data_cleaner.py` Data pipeline

Normalizes raw Monday.com board records before they reach Claude — standardizing currency and date formats, treating blanks/nulls/NaN consistently, and enriching records with derived fields. Exposes `clean_work_order` , `clean_deal` , and `clean_and_enrich` , which also produces the quality report consumed by the prompt layer.

`api/index.py` Vercel entrypoint

Thin adapter that imports the FastAPI `app` object from `main.py` so it can run as a Vercel Python serverless function. Routing to this file is configured in `vercel.json` .

`templates/index.html` Frontend

The entire single-page chat interface — HTML, CSS, and JavaScript in one file, no build step. Renders the chat thread and a live right-hand panel showing every Monday.com/Claude API call as it happens.

`specmatic/` Contract testing

A full Specmatic-based contract-testing suite: an OpenAPI spec of the real endpoints, example fixtures per endpoint, stub responses for mock-server testing, and a dedicated resiliency suite covering malformed input, injection attempts, boundary-length strings, and missing/extra fields.

4 Request Lifecycle

1. Frontend sends `POST /query {query, session_id}`



2. `main.py` fetches Work Orders + Deals boards live via `monday_gql()`



3. `data_cleaner.clean_and_enrich()` normalizes records, builds `quality_report`



4. `build_memory_ctx()` injects last 5 Q&A pairs for this `session_id`



5. `ask_llm()` assembles system prompt (data + quality notes + memory) → calls Claude



6. Response + full API trace returned as JSON → rendered in chat + trace panel

5 API Surface

Method	Path	Description
--------	------	-------------

GET	/	Serves the chat UI (templates/index.html)
POST	/query	Submit a BI question; triggers live Monday.com fetch + Claude analysis
GET	/health	Reports configuration status (API keys present, boards reachable)
GET	/memory	View the stored conversation memory for a session
DELETE	/memory	Clear conversation memory for a session

6 External Dependencies

Data & AI services

- **Monday.com GraphQL API** — live board data (Work Orders, Deals), no caching layer
- **Anthropic Claude API** (claude-opus-4-5) — the BI reasoning engine
- **Groq API** — auxiliary LLM access

Python packages

- `fastapi` + `uvicorn` — async web framework/server
- `httpx` — async HTTP client for Monday.com/Claude calls
- `pydantic` — request/response validation
- `python-dotenv` — environment variable loading
- `groq` — Groq SDK

7 Environment Configuration

Required variables, defined in `.env.example` and set in `.env` locally or in Vercel's dashboard for production:

Variable	Purpose
<code>MONDAY_API_KEY</code>	Auth token for the Monday.com GraphQL API
<code>MONDAY_BOARD_WO</code>	Board ID for the Work Orders board
<code>MONDAY_BOARD_DEALS</code>	Board ID for the Deals board
<code>GROQ_API_KEY</code>	Auth token for Groq LLM access
<code>ANTHROPIC_API_KEY</code>	Auth token for the Claude API

8 Deployment Topology

```
GitHub repo —push→ Vercel —builds→ api/index.py (serverless fn)
|
├─ routes all paths per vercel.json
├─ serves static/ + templates/
└─ reads env vars from Vercel dashboard
```

Note: Vercel serverless functions have execution time limits — if Monday.com or Claude calls are slow, a different host may be needed for production use. Secrets should live in Vercel's environment variable store, never in a committed `.env`.

9 Where Specmatic Is Used

Specmatic is used for **contract-driven testing** of the FastAPI backend — validating that the running app actually matches its own OpenAPI contract, rather than relying on manual happy-path testing. It's wired into the project in three layers:

9.1 — The contract itself

- `specmatic/openapi.yaml` — the OpenAPI spec that defines the "source of truth" for every real endpoint (`/query` , `/health` , `/memory`), including the correct status codes (200 / 502 / 503 / 422) and example request/response bodies.
- `specmatic.yaml` (root) — the Specmatic runner config that points at the spec.
- `specmatic/openapi_examples/` — worked examples per endpoint (e.g. `post_query_monday_timeout_502.json` , `get_health_groq_not_configured.json`) that Specmatic uses to generate realistic test scenarios.

9.2 — Running it against the live app

```
# Terminal 1 – start the app
python -m uvicorn main:app --reload --port 8000

# Terminal 2 – run contract tests against it
docker run --rm -v "$(pwd)":/specmatic -w /specmatic \
  specmatic/specmatic test --testBaseUrl http://host.docker.internal:8000
```

Specmatic reads the OpenAPI spec, auto-generates test scenarios for every status code and edge case it declares, fires them at the running FastAPI app over real HTTP, and fails the build if the app's actual behavior diverges from the contract.

9.3 — Two real bugs it caught in `main.py`

Bug A — Error responses silently returned HTTP 200

Every error path in the `/query` handler returned `JSONResponse({error: true})` with no `status_code` , so it defaulted to **200 OK** even when Monday.com or Groq had failed. Specmatic's generated 502/503 scenarios failed against this, exposing the bug. Fix: explicit `status_code=503` for config errors (missing/failed API keys) and `status_code=502` for upstream failures (Monday.com/Groq call errors), applied at 6 call sites in `main.py` .

Bug B — `session_id` accepted but never actually used

The API accepted a `session_id` field on every request, but conversation memory was stored in a single global list shared by everyone — so two different sessions could read and pollute each other's conversation history. The OpenAPI spec's `/memory` parameters implied per-session isolation that the code didn't enforce. Fix: memory became a `dict[str, list]` keyed by `session_id` , with `build_memory_ctx` , `/memory` GET, and `/memory` DELETE all scoped to the requesting session (7 call sites updated).

Note on scope: Specmatic catches HTTP/contract-level bugs (wrong status codes, malformed responses) — it does not verify business logic like session isolation on its own. Bug B was confirmed with a separate custom `httpx`-based scenario test (`test_sessions_are_isolated`) that asserts two sessions never see each other's data.

9.4 — Resiliency / edge-case coverage

`specmatic/resiliency_tests.yaml` plus `specmatic/stubs/resiliency/` extend testing beyond the happy path, covering:

- Empty query strings and null required fields (expecting `422`)
- Wrong field types (e.g. integer instead of string)
- Extra/unexpected fields in the request body (should be ignored, not rejected)
- Missing optional `session_id` (should default to `"default"`)
- Missing `Content-Type` header

- Very long / boundary-length query strings
- Injection-attempt payloads in the query field

9.5 — Result

Test	Before fix	After fix
GET /health	Pass	Pass
POST /query — 200 success path	Pass	Pass
POST /query — 502 upstream error path	Fail (got 200)	Pass
POST /query — 503 config error path	Fail (got 200)	Pass
POST /query — 422 malformed request	Pass	Pass
GET /memory (session-scoped)	Fail (returned all sessions)	Pass
DELETE /memory (session-scoped)	Fail (cleared all sessions)	Pass

10 Key Design Decisions

Decision	Rationale
No caching — every query hits Monday.com live	Founders need real-time accuracy over lower latency
In-memory session storage (no DB)	Sufficient for a 5-turn memory window at prototype scale
Plain HTML/CSS/JS frontend, no React	No build step; simple enough for a single-page chat UI
Visible API/tool trace in the UI	Builds user trust by showing exactly where each number came from
Claude Opus for analysis	Best reasoning quality on messy, inconsistent business data